NAME:-SRAWAN VITHAL BUCCHE

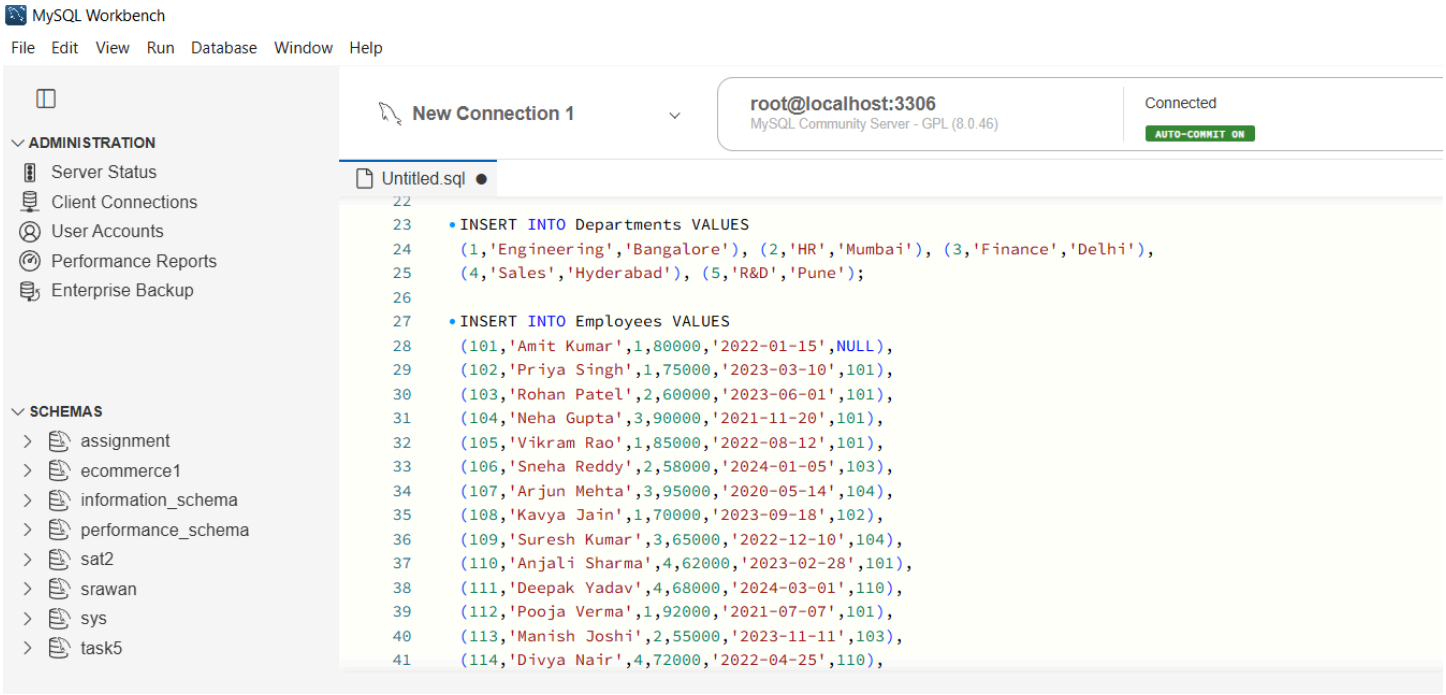
USN:-2JI23CS169

COLLEGE:-JAIN COLLEGE OF ENGINEERING BELAGAVI

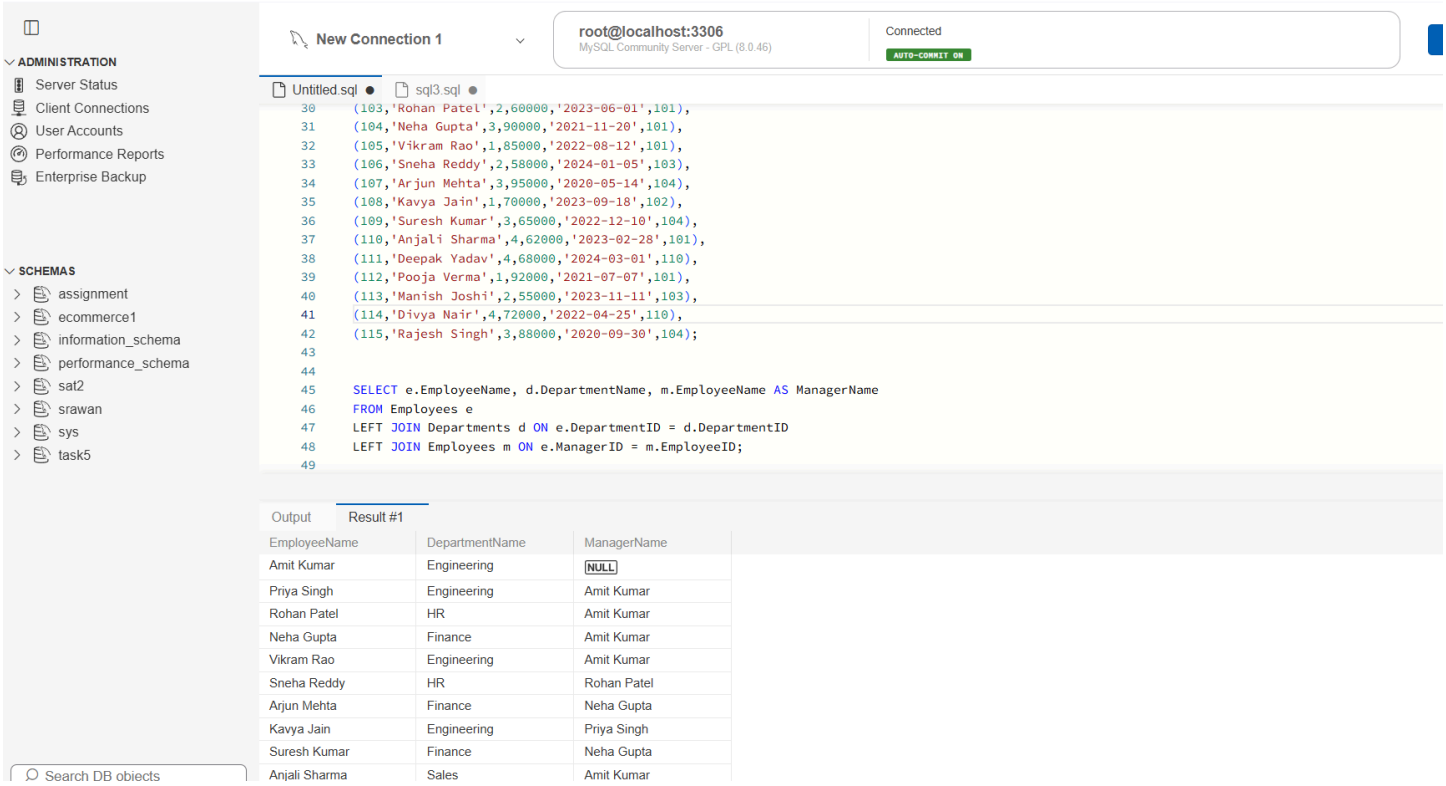
BATCH:-2023-2027

Question 1: Database Design and Advanced Queries

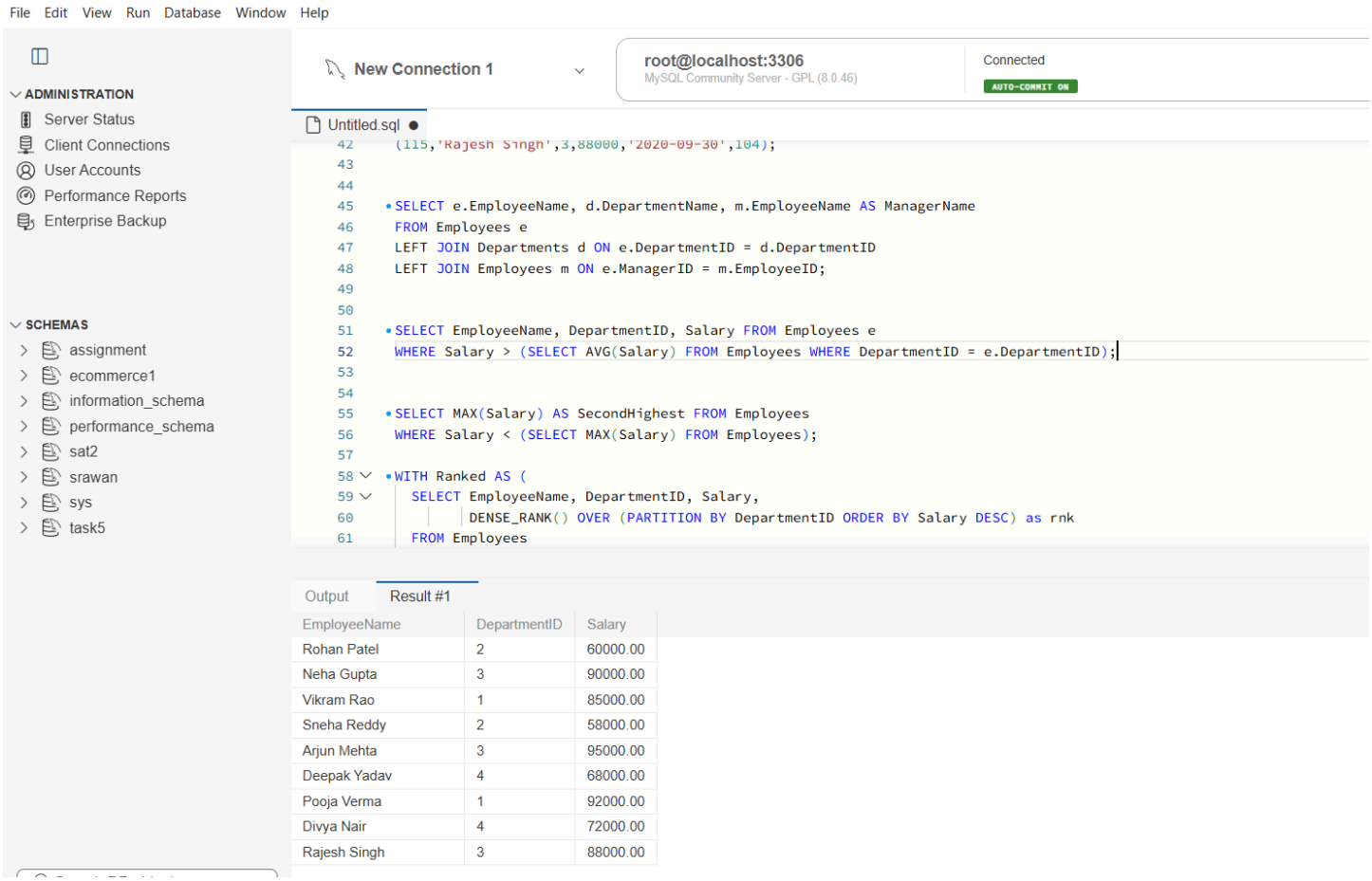




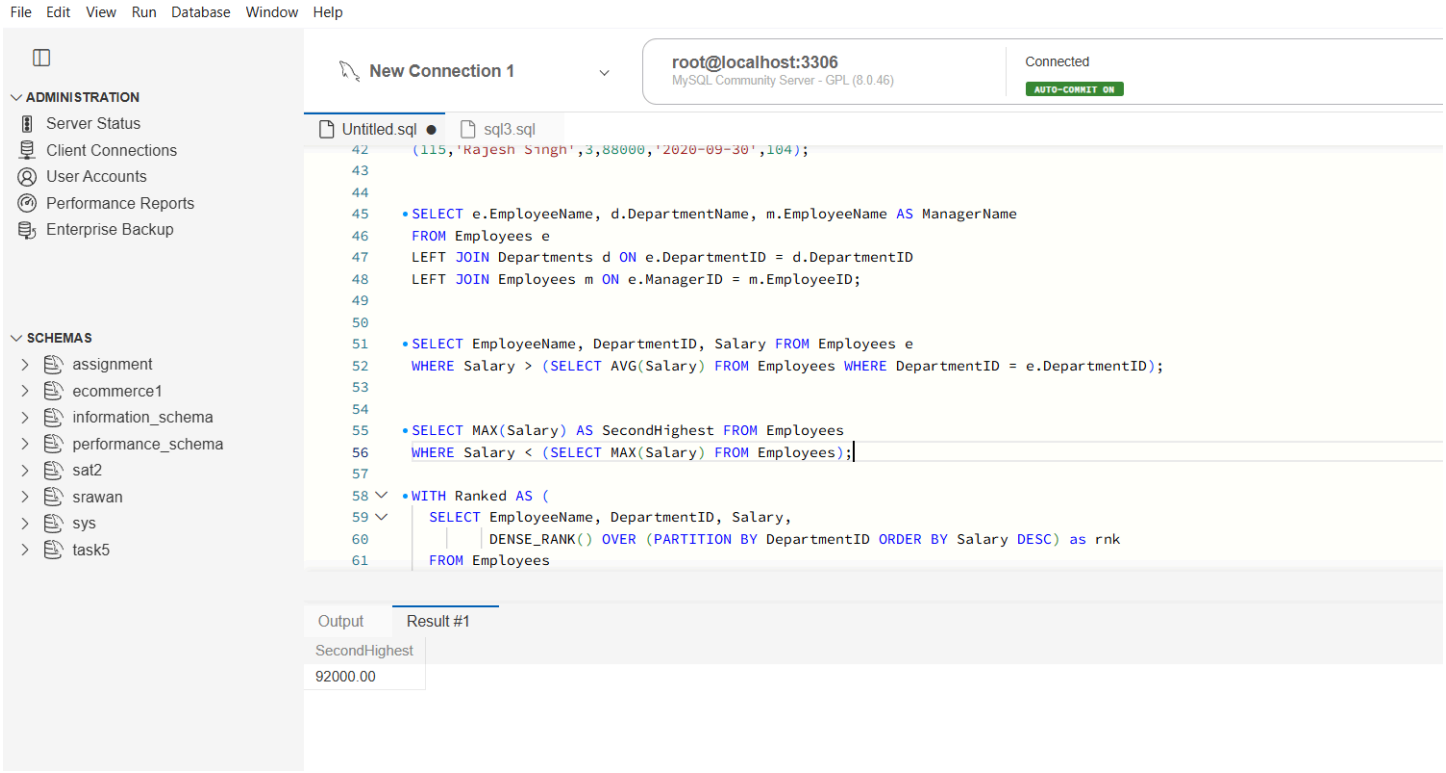
a. Display employee names, department names, and manager names using joins. (2 marks)



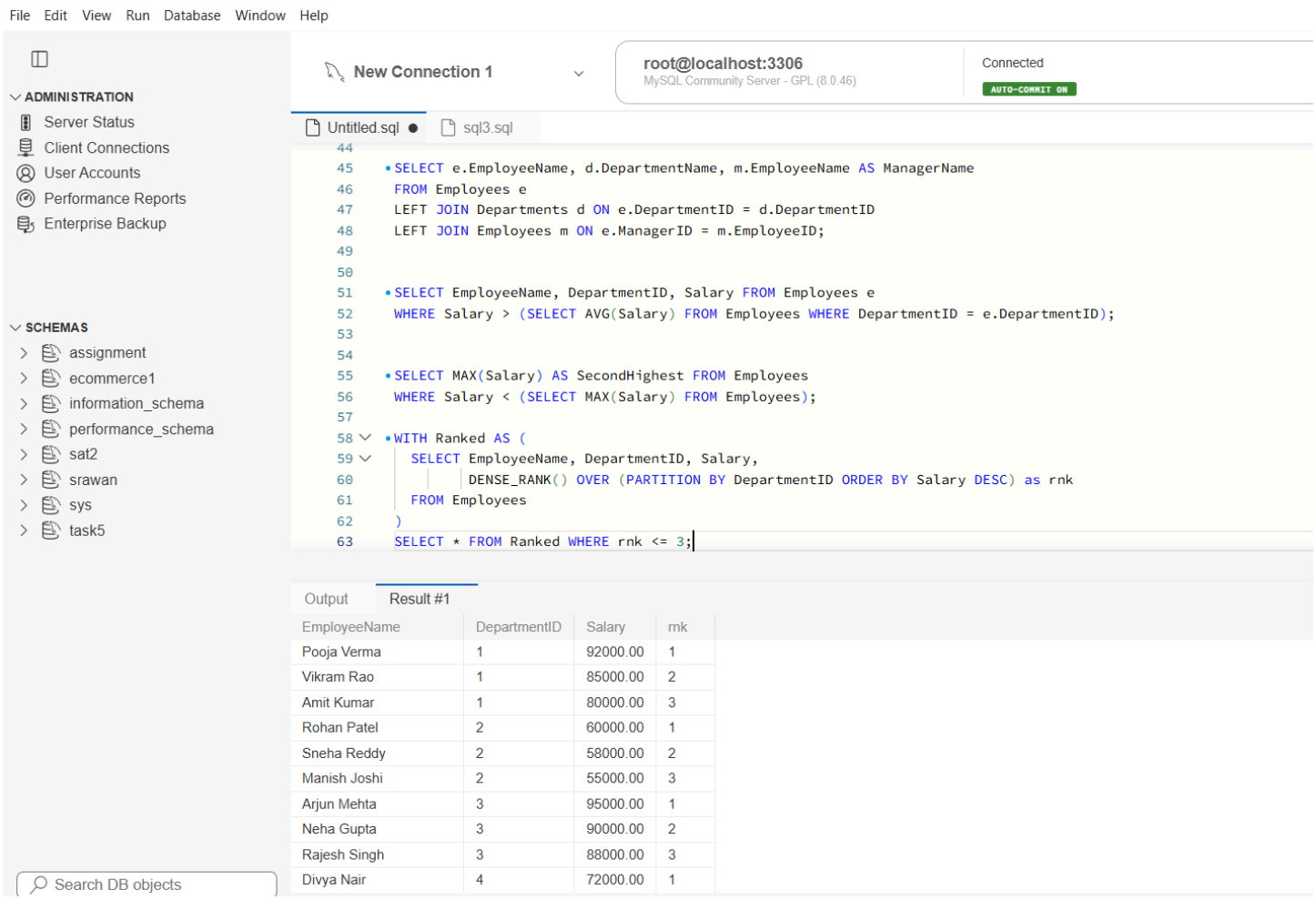
b. Find employees earning more than their department's average salary. (2 marks)



c.

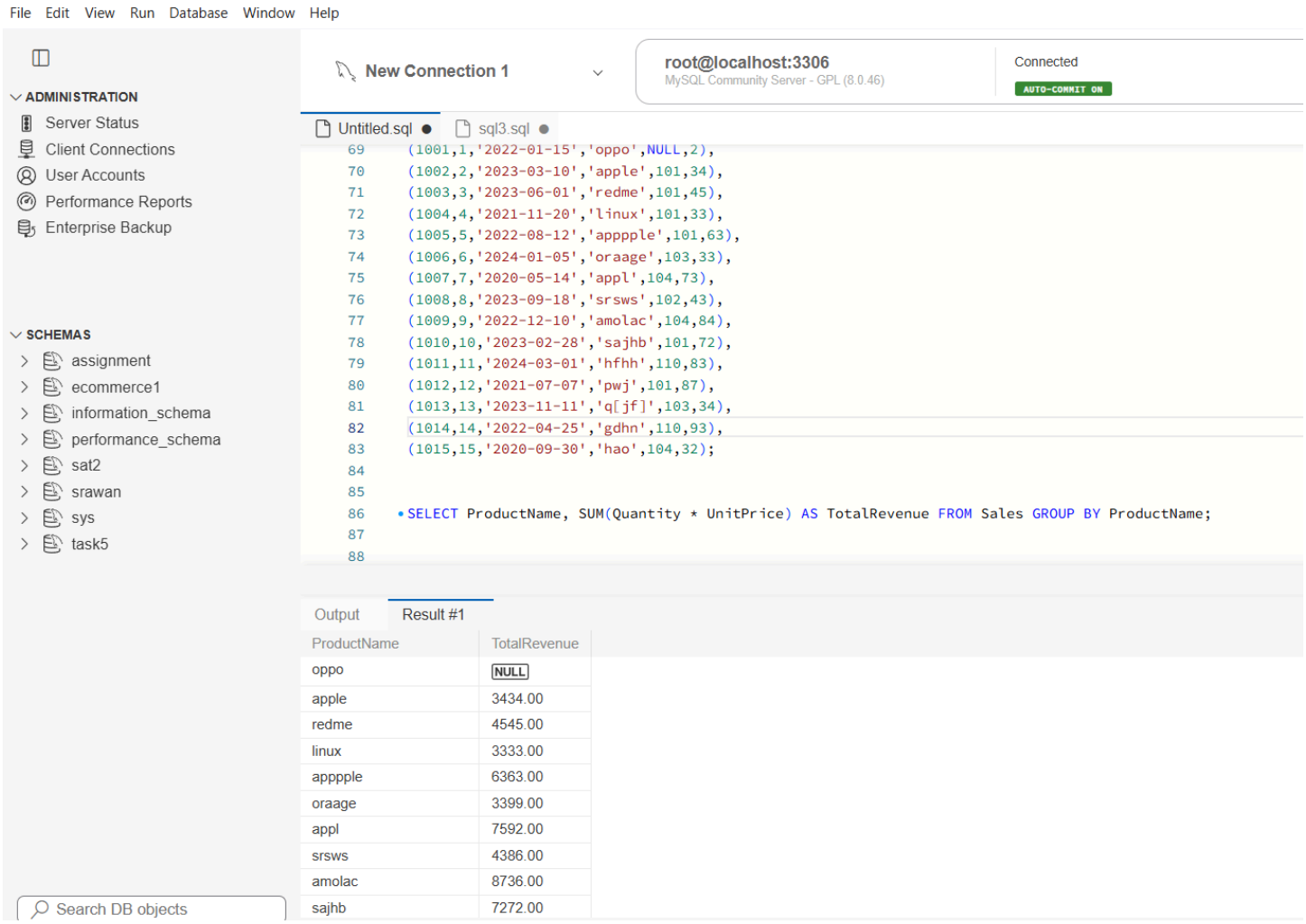

File Edit View Run Database Window Help
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
assignment
ecommerce1
information_schema
performance_schema
sat2
srawan
sys
task5
New Connection 1
root@localhost:3306
MySQL Community Server - GPL (8.0.46)
Connected
AUTO-COMMIT ON
Untitled.sql
sql3.sql
(115,'Rajesh Singh',3,88000,'2020-09-30',104);
SELECT e.EmployeeName, d.DepartmentName, m.EmployeeName AS ManagerName
FROM Employees e
LEFT JOIN Departments d ON e.DepartmentID = d.DepartmentID
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
SELECT EmployeeName, DepartmentID, Salary FROM Employees e
WHERE Salary > (SELECT AVG(Salary) FROM Employees WHERE DepartmentID = e.DepartmentID);
SELECT MAX(Salary) AS SecondHighest FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);
WITH Ranked AS (
SELECT EmployeeName, DepartmentID, Salary,
DENSE_RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC) as rnk
FROM Employees
Output
Result #1
SecondHighest
92000.00

d.

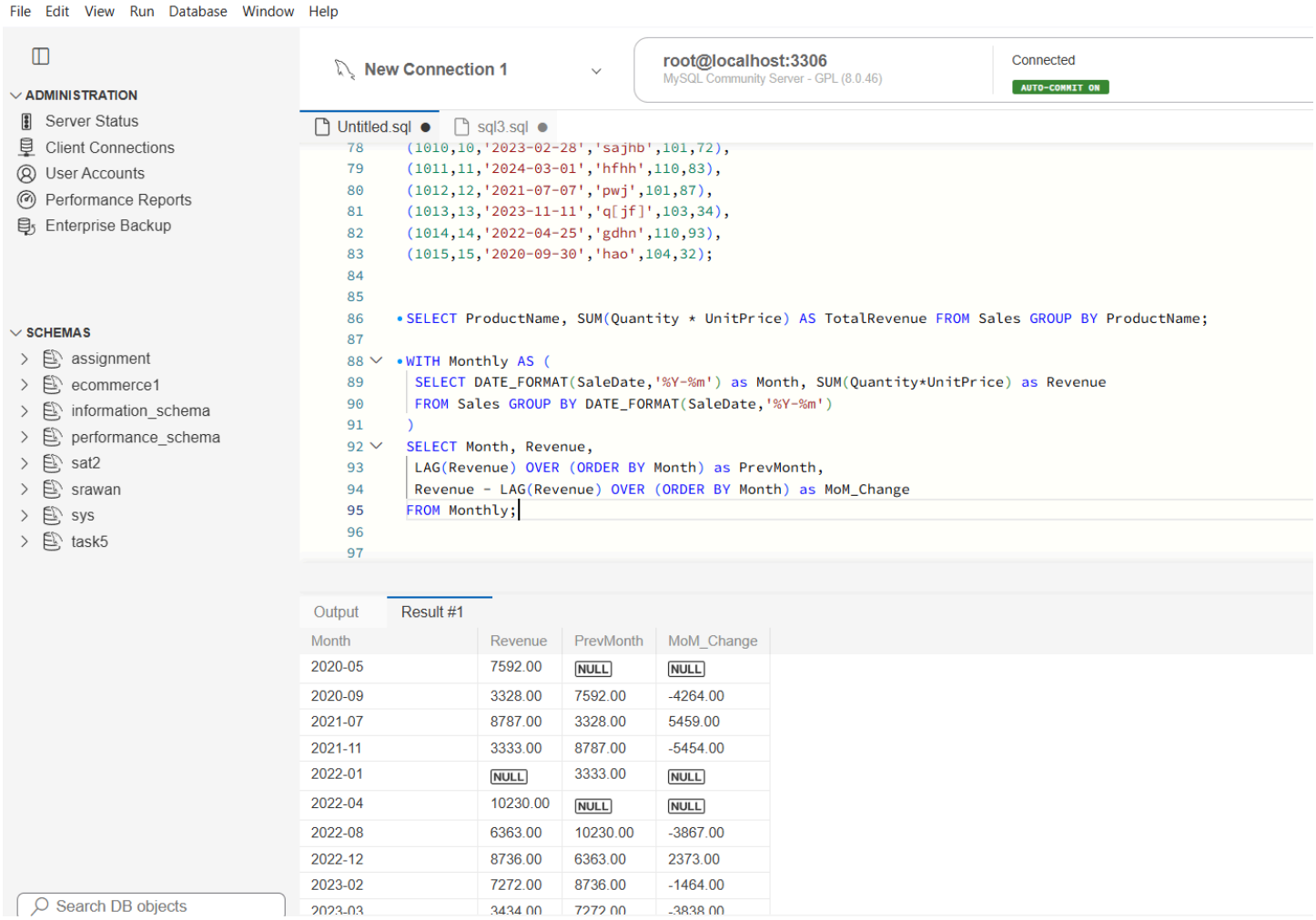

File Edit View Run Database Window Help
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
assignment
ecommerce1
information_schema
performance_schema
sat2
srawan
sys
task5
Search DB objects
New Connection 1
root@localhost:3306
MySQL Community Server - GPL (8.0.46)
Connected
AUTO-COMMIT ON
Untitled.sql
sql3.sql
SELECT e.EmployeeName, d.DepartmentName, m.EmployeeName AS ManagerName
FROM Employees e
LEFT JOIN Departments d ON e.DepartmentID = d.DepartmentID
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
SELECT EmployeeName, DepartmentID, Salary FROM Employees e
WHERE Salary > (SELECT AVG(Salary) FROM Employees WHERE DepartmentID = e.DepartmentID);
SELECT MAX(Salary) AS SecondHighest FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);
WITH Ranked AS (
SELECT EmployeeName, DepartmentID, Salary,
DENSE_RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC) as rnk
FROM Employees
)
SELECT * FROM Ranked WHERE rnk <= 3;
Output
Result #1
EmployeeName DepartmentID Salary rnk
Pooja Verma 1 92000.00 1
Vikram Rao 1 85000.00 2
Amit Kumar 1 80000.00 3
Rohan Patel 2 60000.00 1
Sneha Reddy 2 58000.00 2
Manish Joshi 2 55000.00 3
Arjun Mehta 3 95000.00 1
Neha Gupta 3 90000.00 2
Rajesh Singh 3 88000.00 3
Divya Nair 4 72000.00 1

Question 2: Sales Analytics and Query Optimization (6 Marks) Create a Sales table with the following columns: • SaleID • CustomerID • SaleDate • ProductName • Quantity • UnitPrice

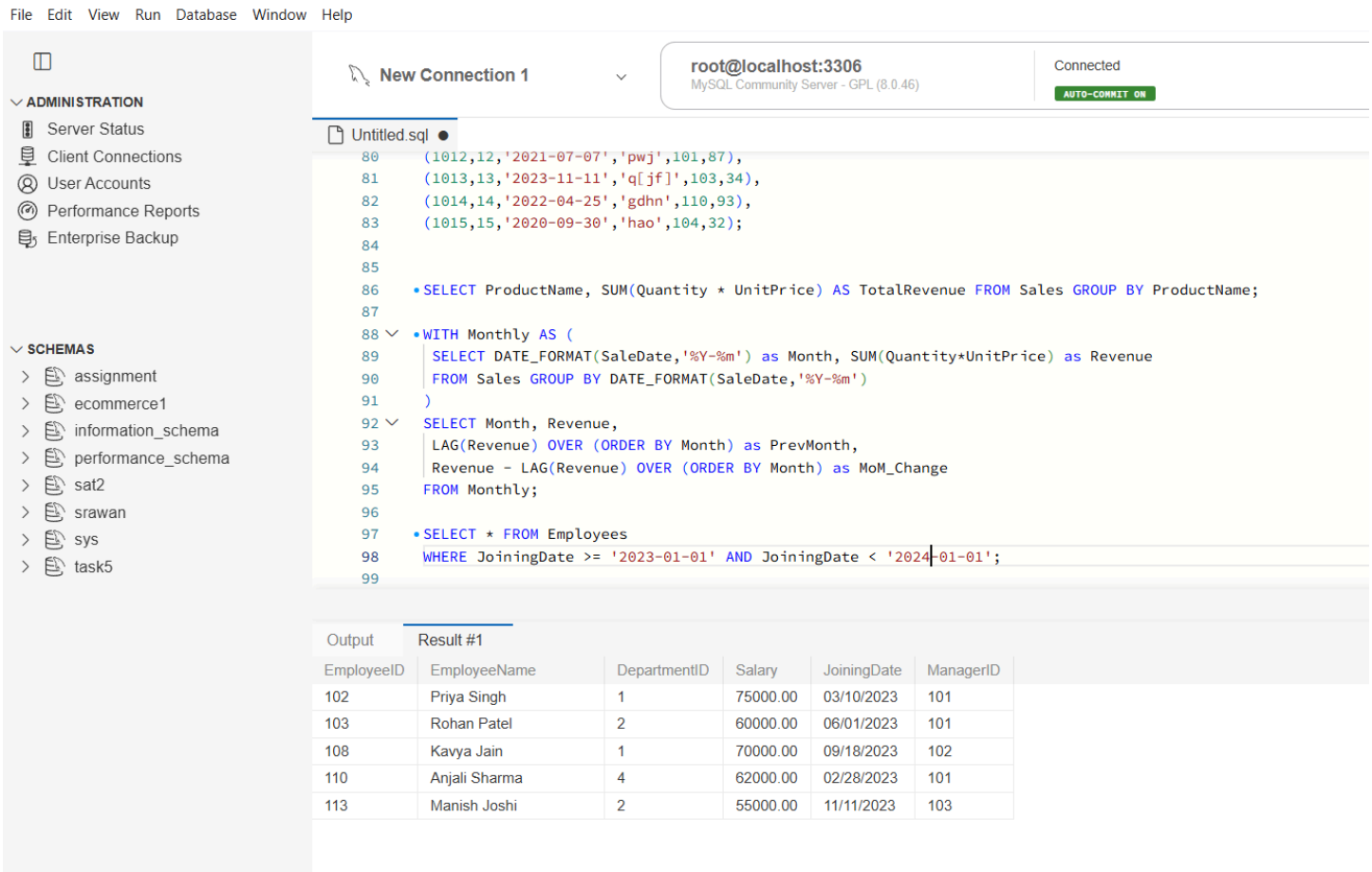
a.



b.


File Edit View Run Database Window Help
New Connection 1
root@localhost:3306
MySQL Community Server - GPL (8.0.46)
Connected
AUTO-COMMIT ON
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
assignment
ecommerce1
information_schema
performance_schema
sat2
srawan
sys
task5
Search DB objects
Untitled.sql
sql3.sql
(1010,10,'2023-02-28','sajhb',101,72),
(1011,11,'2024-03-01','hfhh',110,83),
(1012,12,'2021-07-07','pwj',101,87),
(1013,13,'2023-11-11','q[jf]',103,34),
(1014,14,'2022-04-25','gdhn',110,93),
(1015,15,'2020-09-30','hao',104,32);
SELECT ProductName, SUM(Quantity * UnitPrice) AS TotalRevenue FROM Sales GROUP BY ProductName;
WITH Monthly AS (
SELECT DATE_FORMAT(SaleDate,'%Y-%m') as Month, SUM(Quantity*UnitPrice) as Revenue
FROM Sales GROUP BY DATE_FORMAT(SaleDate,'%Y-%m')
)
SELECT Month, Revenue,
LAG(Revenue) OVER (ORDER BY Month) as PrevMonth,
Revenue - LAG(Revenue) OVER (ORDER BY Month) as MoM_Change
FROM Monthly;
Output
Result #1
Month Revenue PrevMonth MoM_Change
2020-05 7592.00 NULL NULL
2020-09 3328.00 7592.00 -4264.00
2021-07 8787.00 3328.00 5459.00
2021-11 3333.00 8787.00 -5454.00
2022-01 NULL 3333.00 NULL
2022-04 10230.00 NULL NULL
2022-08 6363.00 10230.00 -3867.00
2022-12 8736.00 6363.00 2373.00
2023-02 7272.00 8736.00 -1464.00
2023-03 3434.00 7272.00 -3838.00

C.


File Edit View Run Database Window Help
New Connection 1
root@localhost:3306
MySQL Community Server - GPL (8.0.46)
Connected
AUTO-COMMIT ON
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
assignment
ecommerce1
information_schema
performance_schema
sat2
srawan
sys
task5
Untitled.sql
(1012,12,'2021-07-07','pwj',101,87),
(1013,13,'2023-11-11','q[jf]',103,34),
(1014,14,'2022-04-25','gdhn',110,93),
(1015,15,'2020-09-30','hao',104,32);
SELECT ProductName, SUM(Quantity * UnitPrice) AS TotalRevenue FROM Sales GROUP BY ProductName;
WITH Monthly AS (
SELECT DATE_FORMAT(SaleDate,'%Y-%m') as Month, SUM(Quantity*UnitPrice) as Revenue
FROM Sales GROUP BY DATE_FORMAT(SaleDate,'%Y-%m')
)
SELECT Month, Revenue,
LAG(Revenue) OVER (ORDER BY Month) as PrevMonth,
Revenue - LAG(Revenue) OVER (ORDER BY Month) as MoM_Change
FROM Monthly;
SELECT * FROM Employees
WHERE JoiningDate >= '2023-01-01' AND JoiningDate < '2024-01-01';
Output
Result #1
EmployeeID EmployeeName DepartmentID Salary JoiningDate ManagerID
102 Priya Singh 1 75000.00 03/10/2023 101
103 Rohan Patel 2 60000.00 06/01/2023 101
108 Kavya Jain 1 70000.00 09/18/2023 102
110 Anjali Sharma 4 62000.00 02/28/2023 101
113 Manish Joshi 2 55000.00 11/11/2023 103

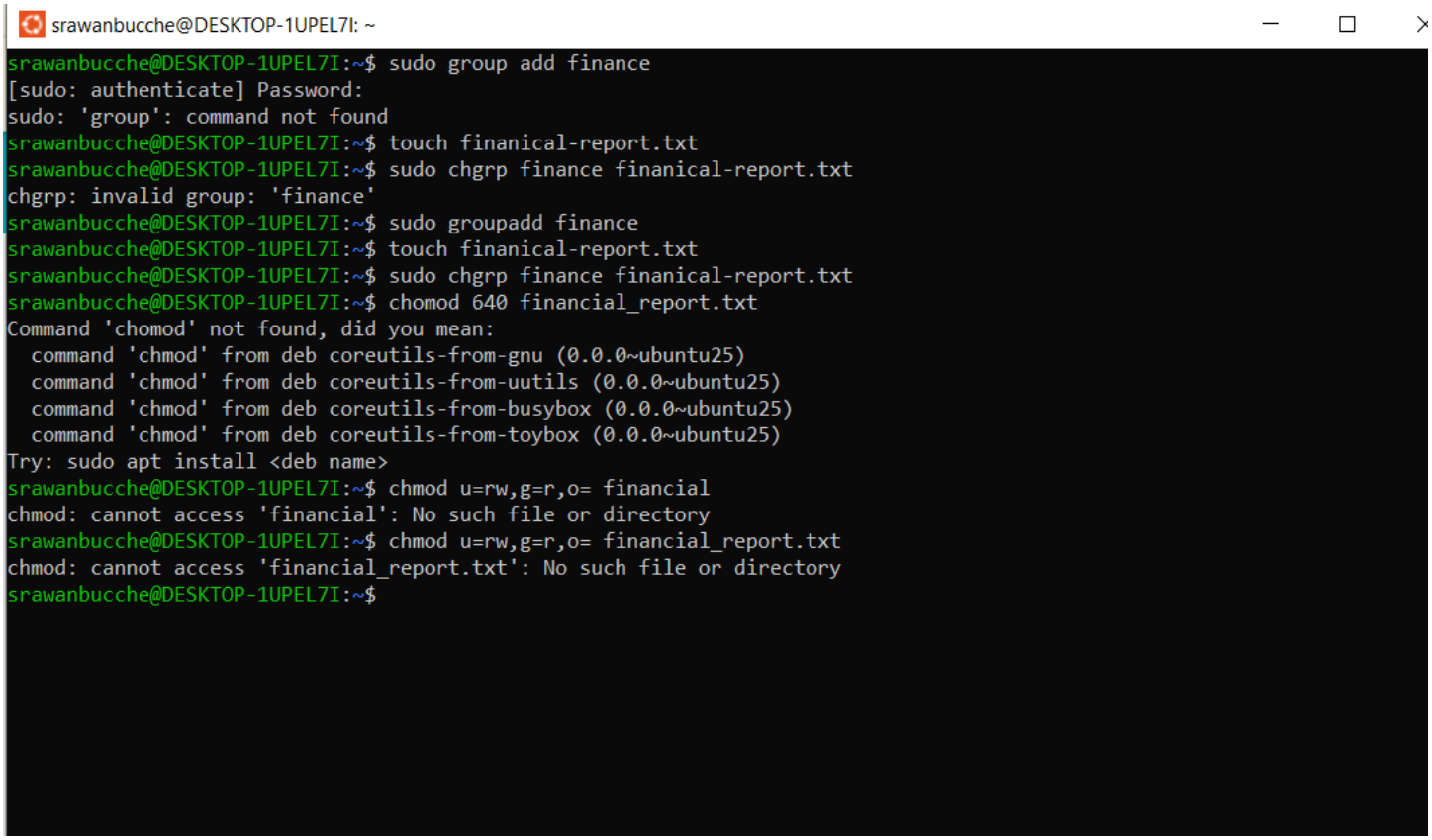
Question 3: Query Performance and Optimization (6 Marks)
(a) Explain how an execution plan helps identify expensive operations in a query. (2 Marks)
Answer:
An execution plan shows how a database executes a query. It helps identify full table scans, inefficient index usage, costly joins, and sorting operations. It also provides estimated costs and row counts to help optimize query performance.

(b) Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning. (2 Marks)
Answer:
A composite index on DepartmentID and Salary is recommended because it improves filtering by department and salary.
SQL Query: CREATE INDEX idx_emp_department_salary ON Employees (DepartmentID, Salary);
This index helps the database locate employees in a specific department and filter them by salary, reducing the number of rows examined.
(c) Write a query to identify duplicate customer email addresses and display only addresses occurring more than once. (2 Marks)
Answer:
SQL Query:
SELECT Email, COUNT*() AS OccurrencesFROM CustomersWHERE Email IS NOT NULLGROUP BY EmailHAVING COUNT()* > 1;
This query groups identical email addresses, counts their occurrences, and displays only those appearing more than once.

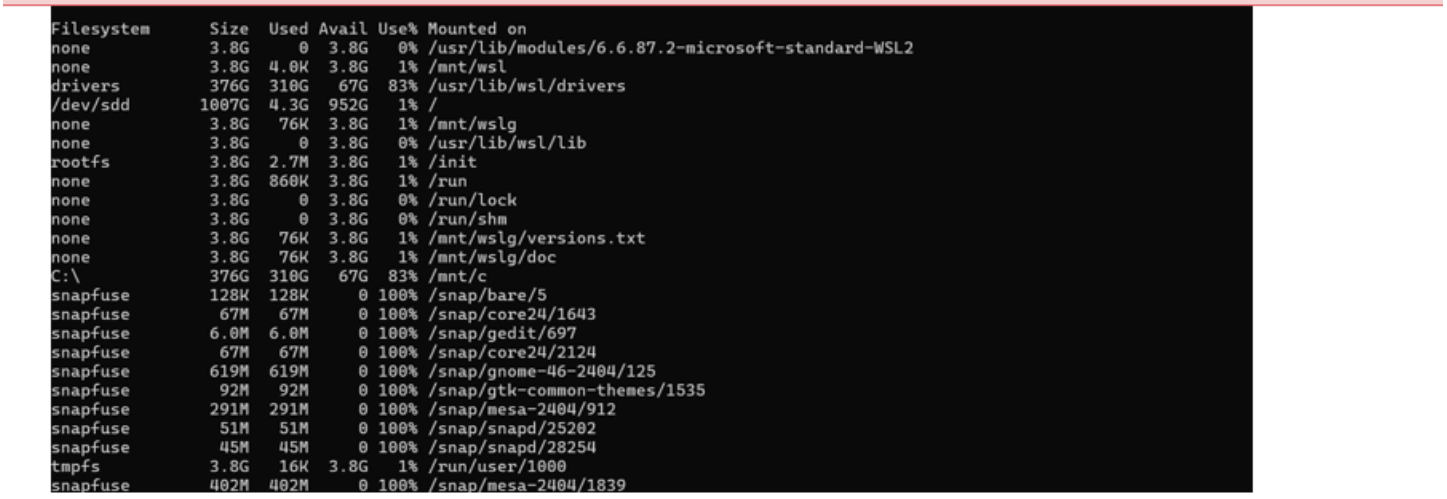
Linux

Question 4: Log Analysis (5 Marks)
(a) Find all lines containing ERROR. (1 Mark)
Answer:
grep 'ERROR' /var/log/application.log
(b) Display the 20 most recent error entries in the file. (1 Mark)
Answer:
grep 'ERROR' /var/log/application.log | tail -n 20
(c) Count the total number of error entries. (1 Mark)
Answer:
grep -c 'ERROR' /var/log/application.log
(d) Extract error entries for a specified date into a separate file. (1 Mark)
Answer:
grep '^2026-10-28 .*ERROR' /var/log/application.log > errors_2026-10-28.log
(e) Explain how you would calculate the number of errors occurring in each hour. (1 Mark)
Answer:
Use awk to extract the date and hour from each error entry and count the occurrences for each hour.
awk '$3 == "ERROR" { hour = substr($2,1,2); count[hour]++ } END { for (h in count) print h ":00 - " count[h] " errors" }' /var/log/application.log
This command assumes the log format has separate date and time fields followed by the log level. It counts error entries for each hour.

Question 5:



```
srawanbucche@DESKTOP-1UPEL7I: ~
srawanbucche@DESKTOP-1UPEL7I:~$ sudo group add finance
[sudo: authenticate] Password:
sudo: 'group': command not found
srawanbucche@DESKTOP-1UPEL7I:~$ touch finanical-report.txt
srawanbucche@DESKTOP-1UPEL7I:~$ sudo chgrp finance finanical-report.txt
chgrp: invalid group: 'finance'
srawanbucche@DESKTOP-1UPEL7I:~$ sudo groupadd finance
srawanbucche@DESKTOP-1UPEL7I:~$ touch finanical-report.txt
srawanbucche@DESKTOP-1UPEL7I:~$ sudo chgrp finance finanical-report.txt
srawanbucche@DESKTOP-1UPEL7I:~$ chomod 640 financial_report.txt
Command 'chomod' not found, did you mean:
  command 'chmod' from deb coreutils-from-gnu (0.0.0~ubuntu25)
  command 'chmod' from deb coreutils-from-uutils (0.0.0~ubuntu25)
  command 'chmod' from deb coreutils-from-busybox (0.0.0~ubuntu25)
  command 'chmod' from deb coreutils-from-toybox (0.0.0~ubuntu25)
Try: sudo apt install <deb name>
srawanbucche@DESKTOP-1UPEL7I:~$ chmod u=rw,g=r,o= financial
chmod: cannot access 'financial': No such file or directory
srawanbucche@DESKTOP-1UPEL7I:~$ chmod u=rw,g=r,o= financial_report.txt
chmod: cannot access 'financial_report.txt': No such file or directory
srawanbucche@DESKTOP-1UPEL7I:~$
```



```
Filesystem      Size  Used Avail Use% Mounted on
none            3.8G     0  3.8G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            3.8G  4.0K  3.8G   1% /mnt/wsl
drivers         376G  310G   67G  83% /usr/lib/wsl/drivers
/dev/sdd       1007G  4.3G  952G   1% /
none            3.8G   76K  3.8G   1% /mnt/wslg
none            3.8G     0  3.8G   0% /usr/lib/wsl/lib
rootfs          3.8G  2.7M  3.8G   1% /init
none            3.8G  860K  3.8G   1% /run
none            3.8G     0  3.8G   0% /run/lock
none            3.8G     0  3.8G   0% /run/shm
none            3.8G   76K  3.8G   1% /mnt/wslg/versions.txt
none            3.8G   76K  3.8G   1% /mnt/wslg/doc
C:\             376G  310G   67G  83% /mnt/c
snapfuse        128K  128K     0 100% /snap/bare/5
snapfuse         67M   67M     0 100% /snap/core24/1643
snapfuse        6.0M  6.0M     0 100% /snap/gedit/697
snapfuse         67M   67M     0 100% /snap/core24/2124
snapfuse        619M  619M     0 100% /snap/gnome-46-2404/125
snapfuse         92M   92M     0 100% /snap/gtk-common-themes/1535
snapfuse        291M  291M     0 100% /snap/mesa-2404/912
snapfuse         51M   51M     0 100% /snap/snapd/25202
snapfuse         45M   45M     0 100% /snap/snapd/28254
tmpfs           3.8G   16K  3.8G   1% /run/user/1000
snapfuse        402M  402M     0 100% /snap/mesa-2404/1839
```

Question 6:

```
 GNU nano 7.2                                  backup_check.sh *

if [ $# -ne 2 ]; then
    echo "Please provide source and destination directories."
    exit 1
fi

SOURCE=$1
DEST=$2

if [ ! -d "$SOURCE" ]; then
    echo "Source directory does not exist."
    exit 1
fi

mkdir -p "$DEST"

DATE=$(date +%Y%m%d_%H%M%S)

tar -czf "$DEST/backup_$DATE.tar.gz" "$SOURCE"

if [ $? -eq 0 ]; then
    echo "Backup successful."
else
    echo "Backup failed."
fi


^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

```
:~$ nano backup_check.sh
:~$ chmod +x backup_check.sh
:~$
:~$ mkdir -p ~/source_data
:~$ echo "Hello Backup" > ~/source_data/test.txt
:~$ ./backup_check.sh ~/source_data ~/backups
ssful: /home/anmol/backups/backup_20261028_051348.tar.gz
:~$ ./backup_check.sh
e provide source and destination directories.
kup_check.sh <source_directory> <destination_directory>
:~$ ./backup_check.sh ~/missing ~/backups
e directory does not exist.
:~$ ls ~/backups

028_051348.tar.gz
:~$
```

SECTION C: PYSPARK DATA ENGINEERING

Question 7.

Question 8.

Question 9.

```
a) schema + 5 rows
df.printSchema()
df.show(5)

 b) >60000
df.filter(col("Salary") > 60000).show()

 c) AnnualSalary
df = df.withColumn("AnnualSalary", col("Salary")*12)

 d) Group by dept
df.groupBy("Department").agg({"Salary":"avg", "EmployeeID":"count"}).show()

 e) Top 5 highest paid
df.orderBy(col("Salary").desc()).limit(5).show()
```

[1]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

spark = SparkSession.builder \
    .appName("Question7") \
    .master("local[*]") \
    .getOrCreate()

data = [
    (1, "Amit", "IT", 75000, "2022-01-10"),
    (2, "Priya", "HR", 45000, "2021-03-15"),
    (3, "Rahul", "IT", 85000, "2020-06-20"),
    (4, "Sneha", "Finance", 65000, "2023-02-01"),
    (5, "Kiran", "HR", 55000, "2022-08-12"),
    (6, "Neha", "IT", 92000, "2019-11-05"),
    (7, "Arjun", "Finance", 70000, "2021-07-18"),
    (8, "Divya", "IT", 60000, "2023-04-10")
]

columns = [
    "EmployeeID", "EmployeeName", "Department",
    "Salary", "JoiningDate"
]

df = spark.createDataFrame(data, columns)
df = df.withColumn("JoiningDate", col("JoiningDate").cast("date"))

print("a. Schema:")
df.printSchema()

print("First five records:")
df.show(5, truncate=False)

print("b. Employees earning more than 60000:")
df.filter(col("Salary") > 60000).show()

print("c. Annual Salary:")
df = df.withColumn("AnnualSalary", col("Salary") * 12)
df.select("EmployeeName", "Salary", "AnnualSalary").show()

print("d. Department-wise average salary and employee count:")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).show()

print("e. Five highest-paid employees:")
df.orderBy(col("Salary").desc()).show(5)
```

```
C:\Users\Anmol\dataEngineer\.venv\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0.
Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
a. Schema:
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)

First five records:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|1         |Amit        |IT        |75000 |2022-01-10 |
|2         |Priya       |HR        |45000 |2021-03-15 |
|3         |Rahul       |IT        |85000 |2020-06-20 |
|4         |Sneha       |Finance   |65000 |2023-02-01 |
|5         |Kiran       |HR        |55000 |2022-08-12 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
b. Employees earning more than 60000:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Amit|        IT| 75000| 2022-01-10|
|         3|       Rahul|        IT| 85000| 2020-06-20|
|         4|       Sneha|   Finance| 65000| 2023-02-01|
|         6|        Neha|        IT| 92000| 2019-11-05|
|         7|       Arjun|   Finance| 70000| 2021-07-18|
+----------+------------+----------+------+-----------+

c. Annual Salary:
+------------+------+------------+
|EmployeeName|Salary|AnnualSalary|
+------------+------+------------+
|        Amit| 75000|      900000|
|       Priya| 45000|      540000|
|       Rahul| 85000|     1020000|
|       Sneha| 65000|      780000|
|       Kiran| 55000|      660000|
|        Neha| 92000|     1104000|
|       Arjun| 70000|      840000|
|       Divya| 60000|      720000|
+------------+------+------------+

d. Department-wise average salary and employee count:
+----------+-------------+-------------+
|Department|AverageSalary|EmployeeCount|
+----------+-------------+-------------+
|        IT|      78000.0|            4|
|        HR|      50000.0|            2|
|   Finance|      67500.0|            2|
+----------+-------------+-------------+

e. Five highest-paid employees:
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|         6|        Neha|        IT| 92000| 2019-11-05|     1104000|
|         3|       Rahul|        IT| 85000| 2020-06-20|     1020000|
|         1|        Amit|        IT| 75000| 2022-01-10|      900000|
|         7|       Arjun|   Finance| 70000| 2021-07-18|      840000|
|         4|       Sneha|   Finance| 65000| 2023-02-01|      780000|
+----------+------------+----------+------+-----------+------------+
only showing top 5 rows
```

```
from pyspark.sql.types import
from pyspark.sql.functions import to_date, col

schema = StructType([
 StructField("TransactionID", StringType()), StructField("CustomerID", StringType()),
 StructField("TransactionDate", StringType()), StructField("Product", StringType()),
 StructField("Quantity", IntegerType()), StructField("UnitPrice", DoubleType())
])

df = spark.read.csv("transactions.csv", header=True, schema=schema)

 b) Remove duplicates by business key TransactionID
df = df.dropDuplicates(["TransactionID"])

 c) Handle missing: Drop if required fields null, or fill
Approach: Product and Quantity are critical, drop if null. UnitPrice fill with avg.
df = df.na.drop(subset=["TransactionID","Product","Quantity"])
df = df.na.fill({"UnitPrice": df.agg({"UnitPrice":"avg"}).collect()[0][0]})

 d) Convert date, handle invalid
df = df.withColumn("TransactionDate", to_date(col("TransactionDate"), "yyyy-MM-dd"))
df = df.filter(col("TransactionDate").isNotNull()) # or keep with flag

 e) Revenue
df = df.withColumn("Revenue", col("Quantity")*col("UnitPrice"))

 f) Total revenue by product desc
df.groupBy("Product").sum("Revenue").orderBy(col("sum(Revenue)").desc()).show()
```

[4]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date, sum
from pyspark.sql.functions import expr


spark = SparkSession.builder.appName("DataCleaning").getOrCreate()

df = spark.read.csv("transactions.csv", header=True, inferSchema=True)

print("Original Data:")
df.show()

# Remove duplicates
df = df.dropDuplicates(["TransactionID"])


df = df.withColumn(
    "TransactionDate",
    expr("try_to_date(TransactionDate, 'yyyy-MM-dd')")
)
```

```
df = df.filter(
    col("CustomerID").isNotNull() &
    col("TransactionDate").isNotNull() &
    col("Quantity").isNotNull() &
    col("UnitPrice").isNotNull()
)

df = df.withColumn("Revenue", col("Quantity") * col("UnitPrice"))

print("Cleaned Data:")
df.show()

print("Revenue by Product:")
df.groupBy("Product").agg(
    sum("Revenue").alias("TotalRevenue")
).orderBy(col("TotalRevenue").desc()).show()
```

```
Original Data:
+-------------------+----------+---------------+--------+--------+---------+
|      TransactionID|CustomerID|TransactionDate| Product|Quantity|UnitPrice|
+-------------------+----------+---------------+--------+--------+---------+
|2026-10-28 01:00:00|        C1|     2026-01-10|  Laptop|       1|    50000|
|2026-10-28 02:00:00|        C2|     2026-01-11|   Mouse|       2|      500|
|2026-10-28 02:00:00|        C2|     2026-01-11|   Mouse|       2|      500|
|2026-10-28 03:00:00|        C3|        invalid|Keyboard|       1|     1500|
|2026-10-28 04:00:00|      NULL|     2026-01-12|  Laptop|       1|    50000|
+-------------------+----------+---------------+--------+--------+---------+

Cleaned Data:
+-------------------+----------+---------------+-------+--------+---------+-------+
|      TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|Revenue|
+-------------------+----------+---------------+-------+--------+---------+-------+
|2026-10-28 01:00:00|        C1|     2026-01-10| Laptop|       1|    50000|  50000|
|2026-10-28 02:00:00|        C2|     2026-01-11|  Mouse|       2|      500|   1000|
+-------------------+----------+---------------+-------+--------+---------+-------+

Revenue by Product:
+-------+------------+
|Product|TotalRevenue|
+-------+------------+
| Laptop|       50000|
|  Mouse|        1000|
+-------+------------+
```

```
customers = spark.read.csv("customers.csv", header=True, inferSchema=True)
orders = spark.read.csv("orders.csv", header=True, inferSchema=True)

 b) Orders with customer details
orders.join(customers, "CustomerID", "inner").show()

 c) Never placed order
customers.join(orders, "CustomerID", "left_anti").show()

 d) Total by city
orders.join(customers, "CustomerID").groupBy("City").sum("OrderAmount").show()

 e) Broadcast join useful when one table is small (<10MB) and large table is huge. Avoids shuffle. Condition: spark.sql.autoBroadcastJoinThreshold. Use hint: broadcast(customers)

 f) Transformations are lazy (map, filter, join) -> builds DAG. Actions trigger execution (show, count, collect).
```

[1]:

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum, broadcast

spark = SparkSession.builder \
    .appName("CustomerOrders") \
    .getOrCreate()

customers_data = [
    (1, "Amit", "Bengaluru"),
    (2, "Priya", "Mumbai"),
    (3, "Rahul", "Delhi"),
    (4, "Sneha", "Bengaluru"),
    (5, "Kiran", "Chennai")
]

orders_data = [
    (101, 1, 5000.0, "2026-01-10"),
    (102, 2, 3000.0, "2026-01-12"),
    (103, 1, 2000.0, "2026-01-15"),
    (104, 3, 7000.0, "2026-01-18"),
    (105, 4, 4000.0, "2026-01-20")
]

customers = spark.createDataFrame(
    customers_data,
    ["CustomerID", "CustomerName", "City"]
)

orders = spark.createDataFrame(
    orders_data,
    ["OrderID", "CustomerID", "OrderAmount", "OrderDate"]
)

print("Customers:")
customers.show()

print("Orders:")
orders.show()
```

```
C:\Users\Anmol\dataEngineer\.venv\Lib\site-packages\pyspark\testing'
0. Some features may not work correctly. It is recommended to use pa
  require_minimum_pandas_version()
Customers:
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|         1|        Amit|Bengaluru|
|         2|       Priya|   Mumbai|
|         3|       Rahul|    Delhi|
|         4|       Sneha|Bengaluru|
|         5|       Kiran|  Chennai|
+----------+------------+---------+

Orders:
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|    101|         1|     5000.0|2026-01-10|
|    102|         2|     3000.0|2026-01-12|
|    103|         1|     2000.0|2026-01-15|
|    104|         3|     7000.0|2026-01-18|
|    105|         4|     4000.0|2026-01-20|
+-------+----------+-----------+----------+
```

```python
joined = orders.join(customers, "CustomerID", "inner")

print("Orders with customer details:")
joined.show()
```

```
Orders with customer details:
+----------+-------+-----------+----------+------------+---------+
|CustomerID|OrderID|OrderAmount| OrderDate|CustomerName|     City|
+----------+-------+-----------+----------+------------+---------+
|         1|    101|     5000.0|2026-01-10|        Amit|Bengaluru|
|         1|    103|     2000.0|2026-01-15|        Amit|Bengaluru|
|         2|    102|     3000.0|2026-01-12|       Priya|   Mumbai|
|         3|    104|     7000.0|2026-01-18|       Rahul|    Delhi|
|         4|    105|     4000.0|2026-01-20|       Sneha|Bengaluru|
+----------+-------+-----------+----------+------------+---------+
```

```
[3]: print("Customers without orders:")

customers.join(
    orders.select("CustomerID").distinct(),
    "CustomerID",
    "left_anti"
).show()
```

```
Customers without orders:
+----------+------------+-------+
|CustomerID|CustomerName|   City|
+----------+------------+-------+
|         5|       Kiran|Chennai|
+----------+------------+-------+
```

```
print("Broadcast join:")

broadcast(orders).join(customers, "CustomerID").show()
```

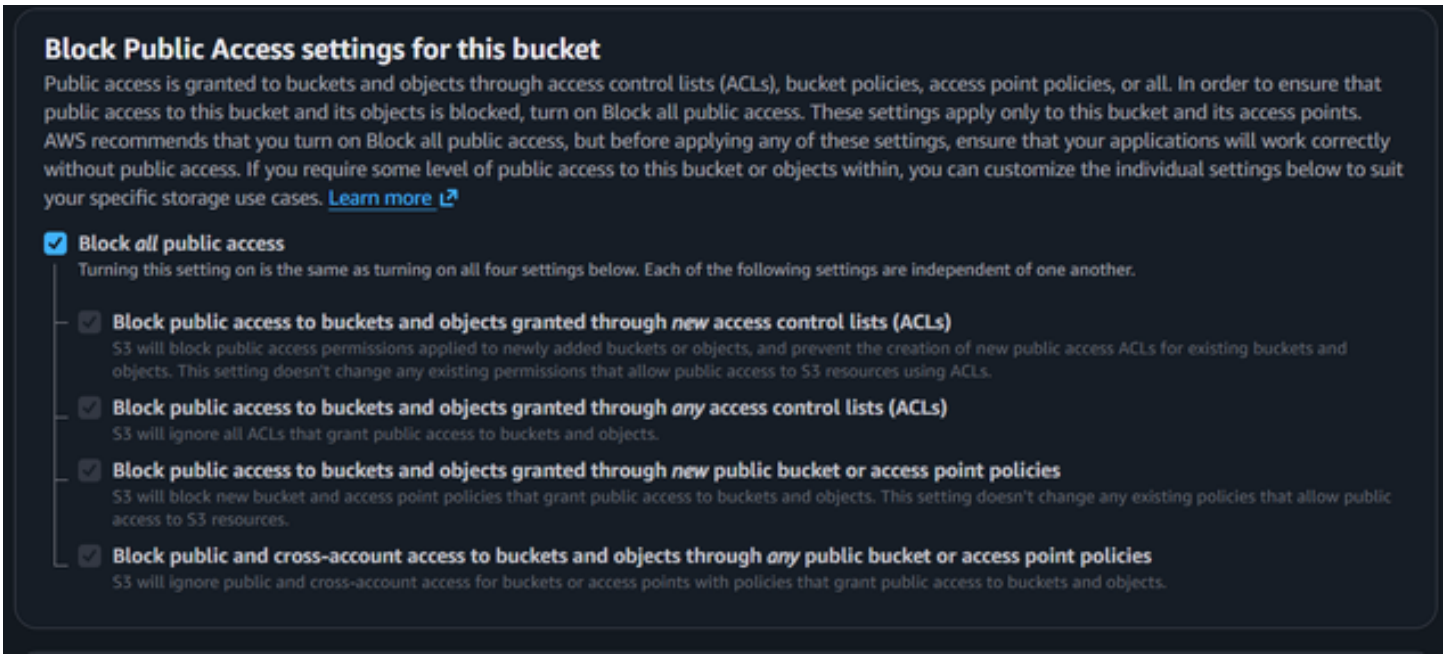
```
Broadcast join:
+----------+-------+-----------+----------+------------+---------+
|CustomerID|OrderID|OrderAmount| OrderDate|CustomerName|     City|
+----------+-------+-----------+----------+------------+---------+
|         1|    103|     2000.0|2026-01-15|        Amit|Bengaluru|
|         1|    101|     5000.0|2026-01-10|        Amit|Bengaluru|
|         2|    102|     3000.0|2026-01-12|       Priya|   Mumbai|
|         3|    104|     7000.0|2026-01-18|       Rahul|    Delhi|
|         4|    105|     4000.0|2026-01-20|       Sneha|Bengaluru|
+----------+-------+-----------+----------+------------+---------+
```

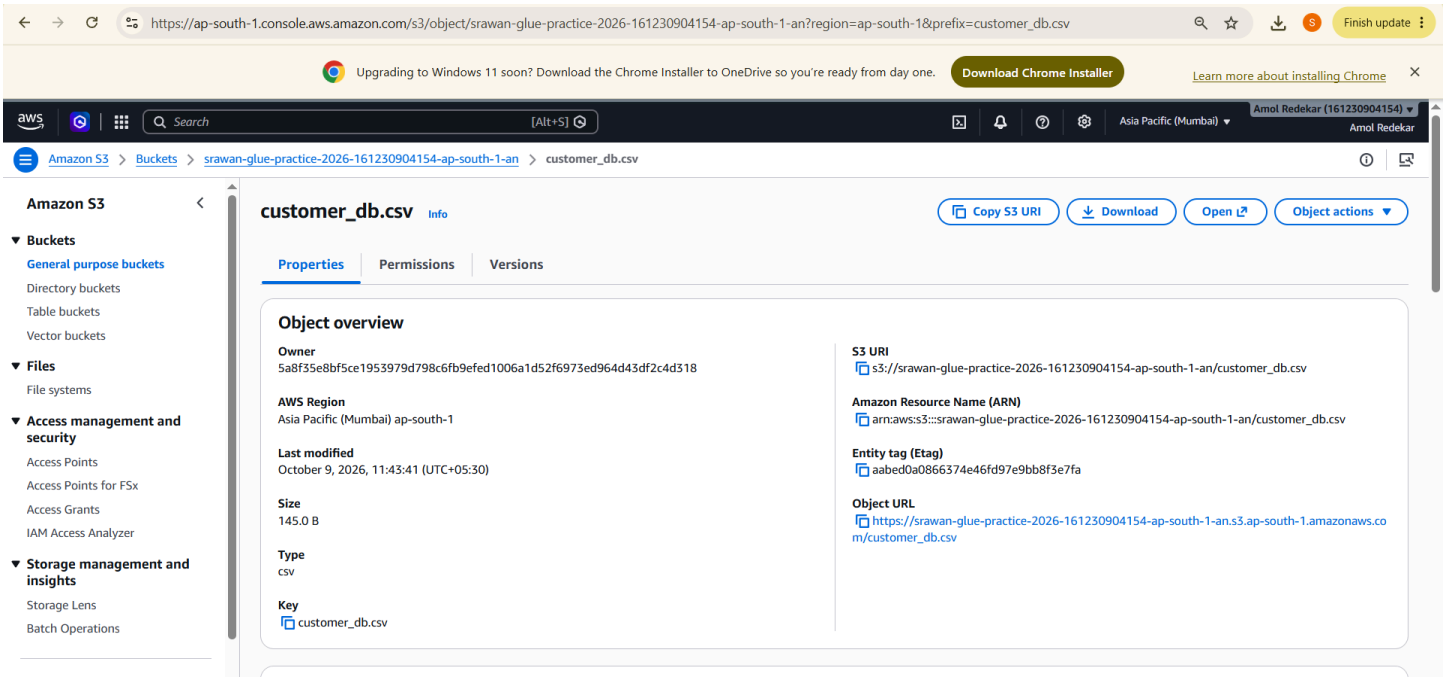
f. Transformations vs. Actions (1 Mark)

- **Transformations:** Create a new DataFrame or execution plan. Example: filter().
- **Actions:** Execute computations and return results. Example: show().

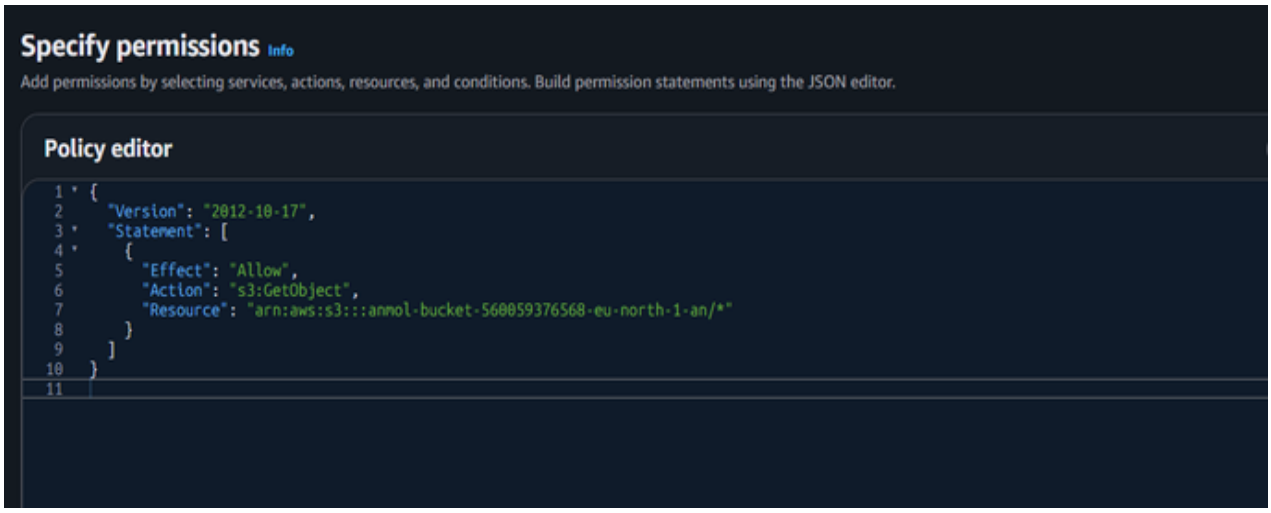
Question 10: S3 and IAM Security (5 Marks)

a. Create an S3 bucket with Block Public Access enabled and upload a sample file. (1 mark)





b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 marks)



c. Explain how you would verify that an unauthorized user cannot read the file. (1 mark)
answer: If an unauthorized user tries to access the file, an Access Denied (403) error will occur. We can verify the failed access attempt using AWS CloudTrail logs (after enabling S3 data events).

d. Explain how encryption at rest protects the stored data. (1 mark)
answer: Encryption at rest protects stored data by encrypting it using encryption keys. Even if an unauthorized person gains access to the storage, they cannot read the data without the required decryption key. In AWS S3, this can be implemented using Server-Side Encryption (SSE).

Question 11: EC2 Troubleshooting and CloudWatch

a. Check the EC2 instance state and status checks.
Open the AWS EC2 console and check whether the instance is in the Running state. Verify that both system status checks and instance status checks are passing.

b. Explain how security groups and route tables can affect application connectivity.
Security groups control incoming and outgoing traffic to the EC2 instance. Incorrect rules may block application traffic. Route tables determine where network traffic is sent, and incorrect routes can prevent users from connecting to the application.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.
Open Amazon CloudWatch and create an alarm for the EC2 instance's CPUUtilization metric. Set the threshold to greater than 85% and define the evaluation period. Configure an SNS notification to alert the administrator when the alarm is triggered.

d. Identify one method for investigating the root cause of high CPU utilization.
Use the EC2 instance's operating system tools, such as the top command in Linux, to identify processes consuming high CPU resources.

Question 12: Event-Driven Processing

A company uploads CSV files to an S3 bucket. Each uploaded file must be validated and processed automatically, with the results stored in another location. Design a solution using AWS services.

a. Identify a suitable service for executing the processing code. (1 mark)

AWS Lambda is suitable for executing code to validate and process uploaded CSV files automatically.

b. Explain how an S3 upload can trigger processing. (1 mark)

Configure an S3 event notification to trigger the Lambda function whenever a new CSV file is uploaded to the input bucket.

c. Explain the IAM permissions needed to read the input file and write the results. (1 mark)

Assign an IAM execution role to Lambda with s3:GetObject permission for the input bucket and s3:PutObject permission for the output bucket.

d. Explain how errors and processing logs can be captured. (1 mark)

Use Amazon CloudWatch Logs to store Lambda execution logs and errors. CloudWatch can also be used to monitor failures.

e. Explain how to prevent duplicate processing or recursive triggering.

Use a separate S3 bucket for the processed results so that output uploads do not trigger the same Lambda function. To prevent duplicate processing, make the function idempotent by tracking processed files.

Question 13: State Machine Design (7 Marks)

a. Identify suitable states for validation, processing, storage, and notification. (2 marks)

Use AWS Step Functions with the following states: Task states for validation, data transformation, sales calculation, S3 storage, and sending the completion notification. Use Choice to check whether the input file is valid and Fail to handle invalid files.

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing. (1 mark)

The Choice state checks the validation result. If the file is valid, it moves to processing. If the file is invalid, it routes the workflow to a Fail state.

c. Explain how Retry and Catch can handle transient failures and permanent errors. (2 marks)

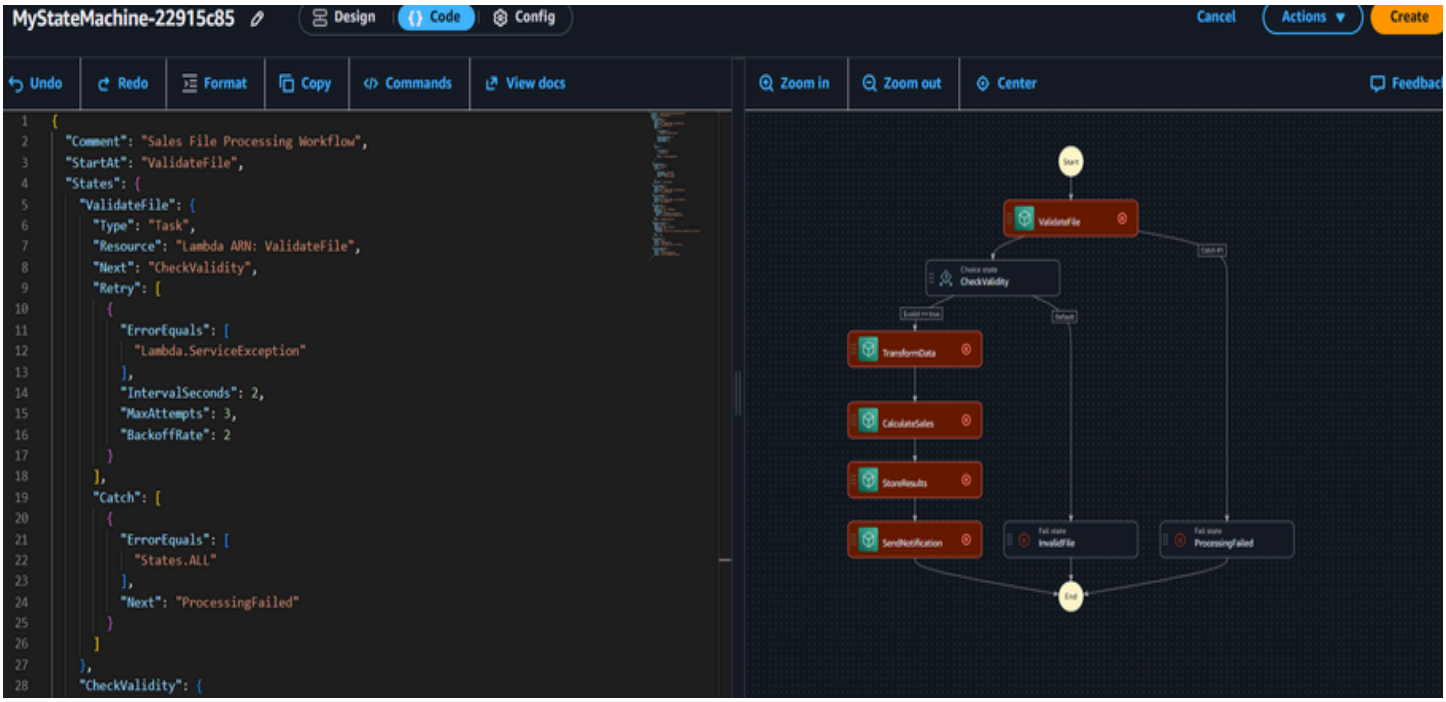
Retry automatically retries temporary failures, such as service errors. Catch handles errors that cannot be recovered by retrying and routes the workflow to an error-handling state or Fail state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services. (1 mark)

Step Functions can invoke AWS Lambda functions using Task states to validate, transform, and calculate sales. It can also use supported AWS service integrations to store results in S3 and send notifications through Amazon SNS.

e. Explain how execution history helps troubleshoot a failed workflow. (1 mark)

Step Functions execution history shows each state's input, output, status, and error details. This helps identify the failed state and troubleshoot the problem.



Question 14: Step Functions Implementation (8 Marks)
A data engineering team has developed three processing functions:

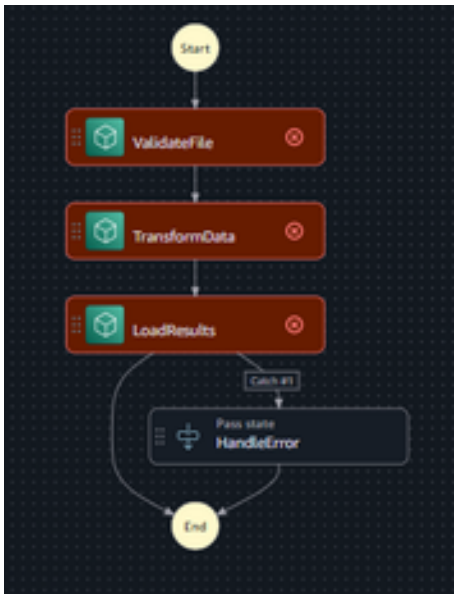
- **ValidateFile**
- **TransformData**
- **LoadResults**

The functions must execute in sequence. Requirements:

- **If validation fails, the workflow must stop.**
- **If transformation fails temporarily, the workflow must retry.**
- **If loading fails permanently, the workflow must record the failure. Tasks:**

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.

```
{
  "Comment": "Data Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "Lambda ARN: ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "Lambda ARN: TransformData",
      "Retry": [
        {
          "ErrorEquals": [
            "Lambda.ServiceException"
          ],
          "IntervalSeconds": 2,
          "MaxAttempts": 3,
          "BackoffRate": 2
        }
      ],
      "Next": "LoadResults"
    },
    "LoadResults": {
      "Type": "Task",
      "Resource": "Lambda ARN: LoadResults",
      "Catch": [
        {
          "ErrorEquals": [
            "States.ALL"
          ],
          "Next": "HandleError"
        }
      ],
      "End": true
    },
    "HandleError": {
      "Type": "Pass",
      "Result": "Loading failed",
      "End": true
    }
  }
}
```



b. Configure a retry policy for the transformation task. (2 marks)
The Retry state retries temporary errors such as Lambda.ServiceException. It waits for 2 seconds, retries up to 3 times, and doubles the wait time after each retry using BackoffRate: 2.

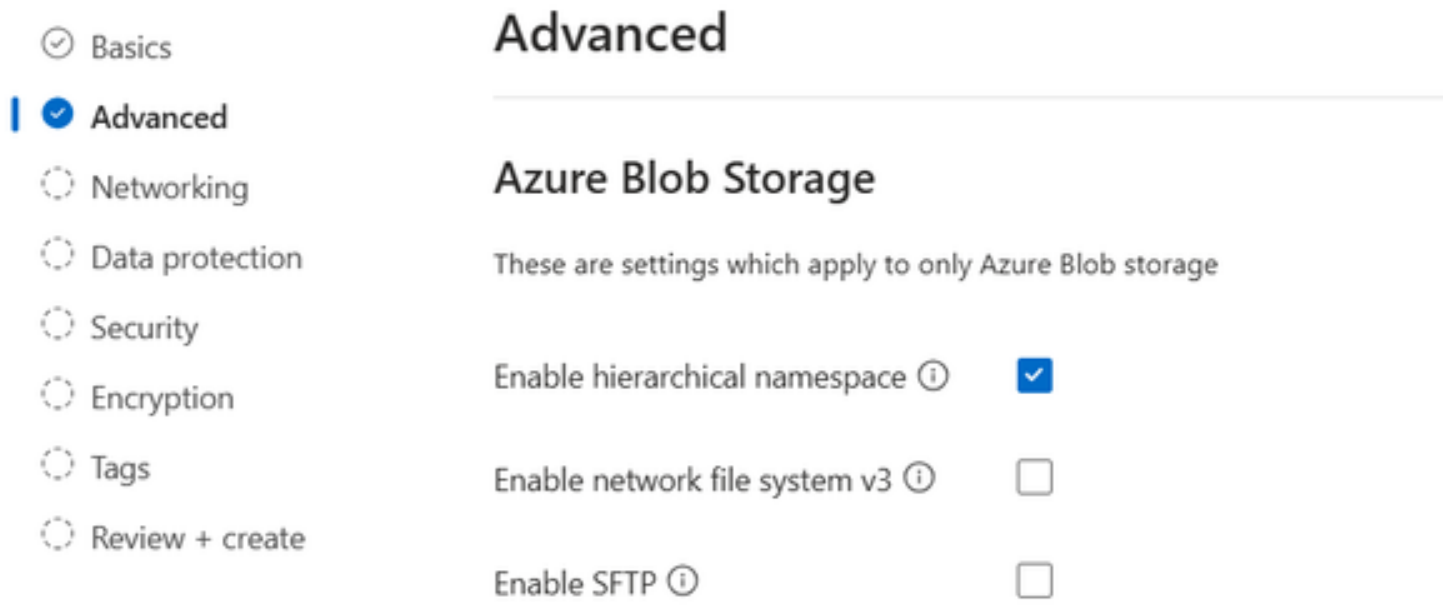
c. Configure a Catch transition for the loading task. (1 mark)
The Catch block captures loading errors and transfers control to the HandleError state, where the failure is recorded.

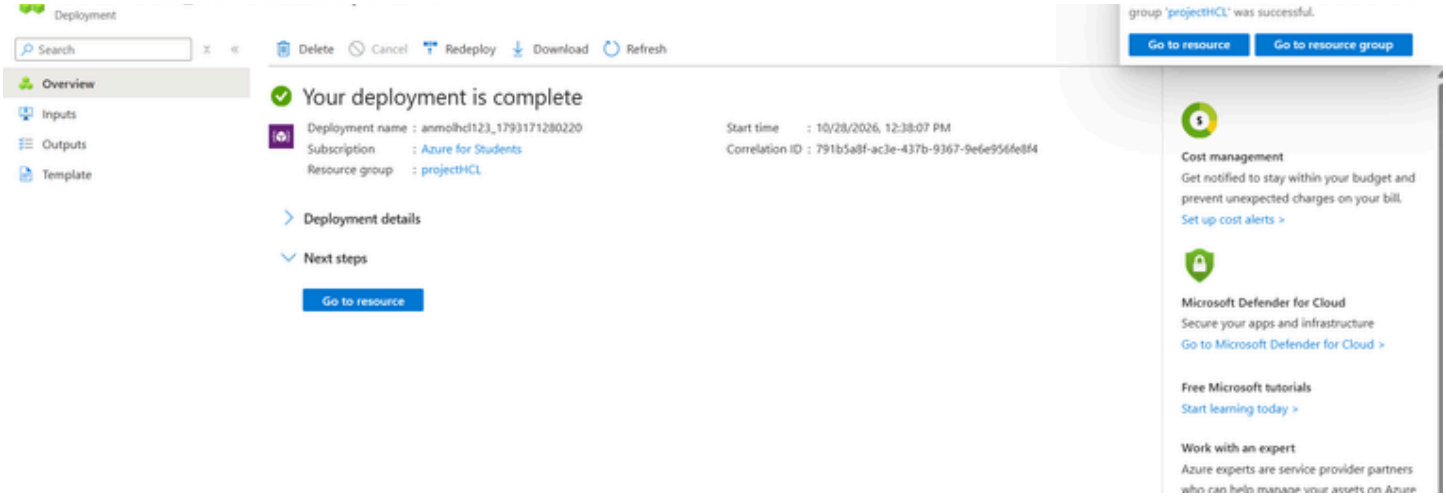
d. Explain how data from one task is passed to the next task. (1 mark)
Step Functions passes the output of one task as the input to the next task by default. JSON data can be used to share processing results between functions.

e. Explain how you would test successful and failed executions. (1 mark)
Run the workflow with valid input to verify successful completion. Test with invalid input and simulated transformation or loading errors to verify that retries and error handling work correctly.

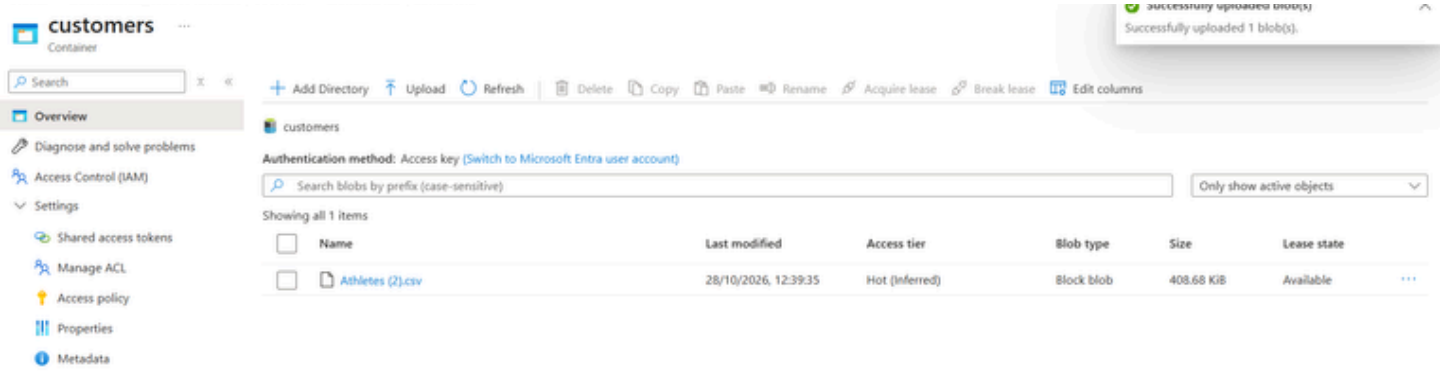
Question 15: Azure Data Lake Storage and Security
a. Configure a storage account with hierarchical namespace enabled. (1 mark)

Open the Azure Portal and create a Storage Account. Select the appropriate subscription and resource group, then enable the Hierarchical namespace option to use Azure Data Lake Storage Gen2.





b. Create a filesystem or container and upload a sample dataset. (1 mark)



Open the storage account, go to Containers, create a container, and upload a sample CSV dataset for analytics.

c. Explain how Microsoft Entra ID and Azure RBAC manage access to the data. (2 marks)

Microsoft Entra ID manages user identities and authentication. Azure RBAC assigns permissions to users or groups based on their roles, such as Storage Blob Data Reader or Storage Blob Data Contributor, to control access to the data.

d. Explain how you would verify that unauthorized users cannot access the dataset. (1 mark)

Sign in as a user without the required permissions and attempt to access or download the dataset. Verify that access is denied, and check the Azure Monitor or diagnostic logs if configured

Question 16:

```
Q16: VM Troubleshooting
a) `az vm get-instance-view -g rg -n vm --query instanceView.statuses`
b) NSG rules: If inbound 22/3389 denied, no connectivity.
c) Subnet UDR missing route to internet or service endpoints.
d) Secure method: Use Azure Bastion or Serial Console or JIT access, not public IP 0.0.0.0/0.
```

Question 17: Azure Data Integration
A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis. Design a solution using Azure services.

a. Select an appropriate service to orchestrate data movement.
Check the VM's status in the Azure portal under Virtual Machines. Verify that the provisioning state is Succeeded and the power state is Running.

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2. (1 mark)
Network Security Group (NSG) rules may block incoming connections if the required port, such as SSH (22) or RDP (3389), is denied. Verify that the inbound rules allow traffic from authorized sources.

c. Describe how to organize files by ingestion date. (1 mark)
Incorrect subnet configurations, route tables, or user-defined routes may prevent traffic from reaching the VM or returning to the client. Check the subnet association, route table, and network connectivity.

d. Explain how to handle malformed records and missing values. (1 mark) e. Explain how to monitor pipeline execution and validate data completeness. (1 mark)
Use Azure Bastion to connect securely to the VM through the Azure portal without assigning a public IP address to the VM. Alternatively, use Azure Serial Console to troubleshoot supported boot and connectivity issues. Restrict NSG rules to authorized sources and avoid opening ports to the entire internet.